All Labs Companion

The SQL and PostgreSQL toolkit behind Labs 1–5 and the hands-on half of the midterm

Advanced Data Management

Covers the Chapter 1 and 4 class examples, Labs 3–5, and Part 3 of the mock midterm

How to read this

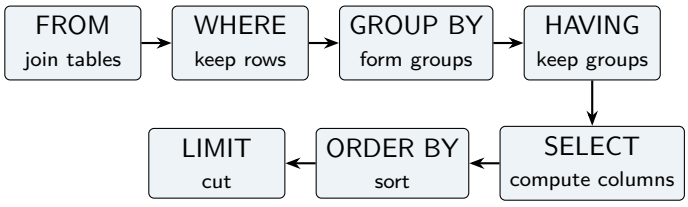
The chapters explain why a plan costs what it costs; the labs and Part 3 of the midterm ask you to *type*. Part I rebuilds the SQL the labs are written in, Part II the PostgreSQL instruments, Part III walks each lab in one page with expected output and exercise answers, Part IV is a write-from-memory drill whose items are the midterm's hands-on questions, with solutions. A cheat sheet closes it. Everything runs on `adm_lab`; drill items D1–D3 rebuild it if you do not have it.

Part I

SQL, rebuilt

1 The shape of a query

```
SELECT   c.country, count(*) AS n, round(avg(o.amount), 2) AS avg_amt  -- 5
FROM     customers c JOIN orders o ON o.cust_id = c.id                 -- 1
WHERE    o.order_date >= date '2025-06-01'                             -- 2
GROUP BY c.country                                                     -- 3
HAVING   count(*) > 1000                                               -- 4
ORDER BY n DESC                                                        -- 6
LIMIT    5;                                                            -- 7
```



Key insight. Written SELECT-first, evaluated FROM-first.

- **WHERE filters rows, HAVING filters groups**: a condition on an aggregate goes in `HAVING`.
- **Aliases**: `AS n` works in `ORDER BY` (after SELECT), not in `WHERE` (before it).
- **Every non-aggregated SELECT column must be in GROUP BY.**

Bottom-up, a plan has the same order: scans and joins, then an aggregate, then Sort and Limit.

2 Joins, grouping, subqueries, CASE

```
SELECT c.name, o.amount FROM customers c JOIN      orders o ON o.cust_id = c.
    id;  -- matches only
SELECT c.name, o.amount FROM customers c LEFT JOIN orders o ON o.cust_id = c.
    id;  -- all customers, NULL amount
SELECT c.name FROM customers c LEFT JOIN orders o ON o.cust_id = c.id
WHERE  o.id IS NULL;                                          -- customers with
     no orders

SELECT country, count(*) FROM customers GROUP BY country HAVING count(*) >
    1800;
SELECT count(*) FILTER (WHERE country = 'MA')::float / count(*) FROM
    customers;  -- selectivity by hand
SELECT ts::date AS day, count(*) FROM events GROUP BY day ORDER BY day;
        -- per-day buckets

SELECT name FROM customers WHERE id IN (SELECT cust_id FROM orders WHERE
    amount > 100);
SELECT amount, CASE WHEN amount < 20 THEN 'small' WHEN amount < 80 THEN '
    medium'
                    ELSE 'large' END AS band FROM orders;
```

Warning / Gotcha: Two silent errors

`count(*) / count(*)` is integer division and returns 0: cast one side `::float`. A `WHERE` on the right table of a `LEFT JOIN` drops the unmatched rows (their columns are NULL); put that condition in the `ON`.

3 Types and casts

```
SELECT '42'::int + 1;                                        -- cast;  CAST('42'
    AS int) is the standard form
SELECT 'Customer ' || 17;                                    -- 'Customer 17'
SELECT date '2025-01-01' + 364;                              -- date + int = days
SELECT now() - interval '30 days';  SELECT current_date - 30;
SELECT timestamp '2026-10-12' - random() * interval '365 days';   -- random
    moment, last year
SELECT round(12.3456::numeric, 2);                           -- round(double, int)
     does not exist
```

```
SELECT floor(random() * 5)::int;                    -- 0..4 as an int (
    array index, id)
```

Types in the labs: `serial` (auto id), `int`, `numeric(8,2)` (money, exact), `double precision` (`random()`, `avg`), `text`/`char(2)`, `date`, `timestamp`, `jsonb` (`payload->>'key'`).

4 DDL, DML and generating data

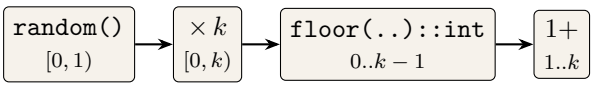
```
CREATE TABLE customers (id serial PRIMARY KEY, name text, country char(2));
CREATE TABLE orders (id serial PRIMARY KEY, cust_id int REFERENCES customers(
    id),
                     product_id int, amount numeric(8,2), order_date date);
ALTER TABLE customers ADD COLUMN city text;
UPDATE customers SET city = CASE WHEN country = 'MA' THEN 'Casablanca' ELSE '
    Other' END;
```

A `PRIMARY KEY` creates a unique B-tree index; `REFERENCES` creates no index on the referencing column (so `orders.cust_id` and `events.user_id` are unindexed until you add one).

Every lab fills its tables with `INSERT ... SELECT` over `generate_series`:

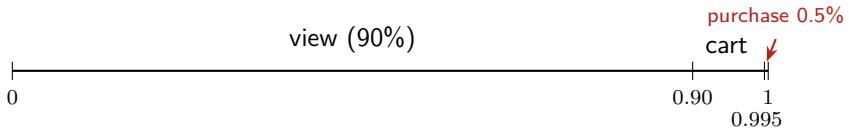
```
INSERT INTO customers (name, country)
SELECT 'Customer ' || g, (ARRAY['MA','FR','ES','US','DE'])[1 + floor(random()
     * 5)::int]
FROM   generate_series(1, 10000) g;

INSERT INTO orders (cust_id, product_id, amount, order_date)
SELECT 1 + floor(random() * 10000)::int,                -- uniform id
    1..10000
       1 + floor(random() * 200)::int,
       round((random() * 100 + 5)::numeric, 2),         -- uniform
    5.00..105.00
       date '2025-01-01' + floor(random() * 365)::int   -- uniform day of
    2025
FROM   generate_series(1, 50000);
ANALYZE;                                                -- always after a
    bulk load
```



`generate_series(1, n)` acts as a table of n rows; the SELECT list is evaluated per row, so each `random()` draws afresh. Array indexing (1-based) gives a uniform categorical value.

4.1 Skew: one draw, several cuts




```
INSERT INTO events (user_id, event_type, ts, product_id, payload, device)
SELECT 1 + floor(random() * 2000000)::int,
       CASE WHEN r < 0.90 THEN 'view' WHEN r < 0.995 THEN 'cart' ELSE '
    purchase' END,
       timestamp '2026-10-12' - random() * interval '365 days',
       1 + floor(random() * 50000)::int, '{}'::jsonb,
       (ARRAY['web','ios','android'])[1 + floor(random() * 3)::int]
FROM  (SELECT random() AS r FROM generate_series(1, 1000000)) g;
```

Warning / Gotcha: Why the subquery

`WHEN random() < 0.90 ...  WHEN random() < 0.995` draws a new number in the second branch: purchase becomes $0.10 \times 0.005 = 0.05\%$, ten times too rare. Name the draw `r` once, compare it twice. (An alias cannot be reused in the same SELECT list, hence the derived table.)

5 The two schemas

`adm_lab` (labs)		
`customers`	`id PK, name, country char(2)` [+ `city`, Lab 5]	10,000–20,000 rows
`orders`	`id PK, cust_id FK, product_id, amount numeric(8,2), order_date`	50,000, grown to 500,000 in Lab 3
`shopstream` (midterm)		
`users`	`user_id PK, email, country, city, segment, signup_ts`	2,000,000 rows, 30,000 pages
`events`	`event_id PK, user_id, event_type, ts, product_id, payload jsonb, device`	50,000,000 rows, 1,000,000 pages, 50 rows/page

`amount` is uniform on 5–105 (`> 90` keeps 15%, `> 104` about 1%); `event_type` is 90/9.5/0.5; `city` determines `country`; shopstream's statistics are stale by 10×.

Part II

The PostgreSQL instruments

6 EXPLAIN, line by line

```
\timing on                                  -- psql: elapsed time per statement
EXPLAIN SELECT ...;                         -- plan and estimates, does not run
     the query
EXPLAIN (ANALYZE, BUFFERS) SELECT ...;      -- runs it: actual rows, times,
    pages
BEGIN; EXPLAIN ANALYZE DELETE ...; ROLLBACK;   -- ANALYZE really executes DML
```

```
Sort  (cost=1031..1036 rows=1996 width=40) (actual time=121.84..122.07 rows=1996
    loops=1)
  Sort Key: c.name       Sort Method: quicksort  Memory: 205kB       Buffers: shared
    hit=3355
  ->  HashAggregate  (actual time=120.93..121.31 rows=1996 loops=1)   Batches: 1
        ->  Hash Join  (actual time=4.02..105.66 rows=50112 loops=1)
              Hash Cond: (o.cust_id = c.id)
              ->  Seq Scan on orders o  (actual time=0.01..37.95 rows=500000 loops=1)
              ->  Hash  (actual rows=2000 loops=1)  Buckets: 2048  Batches: 1  Memory
     Usage: 117kB
                    ->  Seq Scan on customers c  (rows=2000)  Filter: (country = 'MA
    ')
                          Rows Removed by Filter: 18000
Execution Time: 122.38 ms
```

`cost=a..b`	estimate in cost units (sequential page = 1): to first row, to all rows
`rows=` / `actual rows=`	estimated vs actual rows *per loop*; a gap of 10× is a statistics problem
`actual time=a..b`	ms to first and last row; own work ≈ b minus children's b
`loops=`	times the node ran (inner of a Nested Loop: once per outer row); total = rows × loops
`Buffers: shared hit=h read=r`	pages from the pool and from disk; $h + r$ is the chapters' page count
`Filter` / `Rows Removed`	predicate applied while scanning; `Index Cond`: answered by the index
`Sort Method`	`quicksort Memory` (fits `work_mem`) or `external merge Disk`
`Batches > 1`	hash build spilled: Grace hash join

Read it in four moves: find the leaves (scans); walk up each `->`; compare estimate with actual from the leaves up, the first big gap is the root cause; mark the pipeline breakers (`Sort`, `Hash`, `HashAggregate`).

7 Steering, statistics, sizes, buffers, Parquet

```
-- steer (teaching only): a penalty, not a ban
SET enable_hashjoin = off;  SET enable_mergejoin = off;  SET enable_seqscan =
     off;
SET work_mem = '64kB';   RESET work_mem;   RESET ALL;   SHOW work_mem;

-- what the planner believes
SELECT relname, reltuples::bigint, relpages FROM pg_class WHERE relname = '
    orders';
SELECT attname, n_distinct, null_frac, most_common_vals, most_common_freqs,
       histogram_bounds, correlation FROM pg_stats WHERE tablename = 'orders'
    ;
SELECT relname, n_live_tup, last_analyze, last_autoanalyze FROM
    pg_stat_user_tables;
```

```sql
SELECT indexrelname, idx_scan FROM pg_stat_user_indexes WHERE relname = '
    orders';

-- refresh / teach it
ANALYZE orders;                    VACUUM ANALYZE orders;
CREATE STATISTICS s_geo (dependencies) ON country, city FROM customers;
    ANALYZE customers;
ALTER TABLE events ALTER COLUMN event_type SET STATISTICS 1000;

-- sizes and pages
SELECT pg_relation_size('orders') / 8192 AS pages;   -- B (
    pg_total_relation_size adds indexes)
CREATE EXTENSION IF NOT EXISTS pageinspect;
SELECT count(*) FROM heap_page_items(get_raw_page('orders', 0));  -- rows on
    page 0
SELECT ctid, id FROM orders LIMIT 3;                  -- (page, slot) of each
     row

-- buffer pool
CREATE EXTENSION IF NOT EXISTS pg_buffercache;
SELECT c.relname, count(*) AS buffers
FROM   pg_buffercache b JOIN pg_class c ON b.relfilenode =
    pg_relation_filenode(c.oid)
WHERE  b.reldatabase IN (0, (SELECT oid FROM pg_database WHERE datname =
    current_database()))
GROUP BY c.relname ORDER BY buffers DESC LIMIT 10;

-- one column to Parquet (pg_parquet extension, or COPY from DuckDB's
    postgres ext.)
COPY (SELECT ts FROM events) TO '/tmp/events_ts.parquet' (FORMAT parquet);
SELECT ts::date AS day, count(*) FROM read_parquet('/tmp/events_ts.parquet')
     -- DuckDB
GROUP BY day ORDER BY day;
```

Concept: Reading pg_stats

`n_distinct` negative = fraction of rows (-1: unique). `most_common_vals/freqs`: exact frequency for listed values. `histogram_bounds`: equi-depth buckets for ranges. `correlation` near 1: physical order follows the column, cheap range scans and BRIN. The planner never counts rows at plan time; it reads these numbers, and `ANALYZE` is what rewrites them. Autovacuum re-analyses after about 10% of a table changes, so observe stale-statistics effects right after the insert.

Part III

The labs, one page each

8 Chapter 1 class examples (the Lab 1 content)

Technique: What each demo shows

1. **Setup**: the two tables and generators of §4, then `ANALYZE`.
2. **Life of a query**: `EXPLAIN` on the MA join-and-group gives HashAggregate over Hash Join over two Seq Scans; `(ANALYZE, BUFFERS)` adds actual rows and pages.
3. **Declarative vs procedural**: `amount > 100` is a Seq Scan; after `SET enable_seqscan = off` and `CREATE INDEX idx_amount ON orders(amount)` it is an index/bitmap scan. Same SQL, new strategy.
4. **OLTP vs OLAP**: `WHERE id = 4275` touches one row via the PK index; the country aggregate sweeps all.
5. **Row vs column**: `COPY (SELECT * FROM orders) TO '/tmp/orders.parquet' (FORMAT parquet)`, then `SELECT AVG(amount) FROM '/tmp/orders.parquet'` in DuckDB reads one column.
6. **SQL refresher**: top-5 by total (`ORDER BY total DESC LIMIT 5`), `HAVING count(*) > 1800`, an `IN` subquery (plan: a semi-join).
7. **The catalog is data**: `pg_class.reltuples`, `pg_stats.n_distinct` are the optimizer's inputs.

9 Lab 3: Indexing

Technique: Part by part

0. Grow `orders` to 500k rows (`generate_series(1, 450000)`), `ANALYZE`.
1. **Baseline**: both Seq Scan; `Buffers` $\approx B$ pages of the table.
2. **Indexes**: `idx_cust_name` turns the point lookup into an Index Scan (3–4 buffers); `idx_orders_date` turns the ten-day range (3%) into Bitmap Index + Bitmap Heap Scan (still "the index used").
3. **Prefix rule**, `idx_oc(cust_id, order_date)`: `cust_id = 42` uses it, both columns use it, `order_date` alone does not (drop `idx_orders_date` to see the Seq Scan).
4. **Selectivity flip**: `> 104` (1%) index; `> 60` (45%) and `> 20` (85%) Seq Scan. Bisect the threshold; the flip is typically 5–20%. Rule of thumb $s < 1/(4 \cdot \text{rows per page})$ for random fetches.
5. **Expression index**: `lower(name) = ...` ignores `idx_cust_name`; uses `idx_lower_name`.
6. **Write cost**: 100k inserts with three indexes vs none, expect 2–4×

slower (one leaf write per index per row).

Exercises: (1) speedup = seq time / index time, better: buffers ≈ 150 vs 4; (2) queries 1 and 2 used `idx_oc`, 3 did not; (3) report the flip fraction, justify by random vs sequential page cost; (4) two plans; (5) ratio and the leaf-write explanation; (6) index `orders.cust_id` (joins, selective), not `customers.country` (5 values, 20% each, above the flip).

```
CREATE INDEX i ON orders (order_date);                    -- B-tree: =, <,
    BETWEEN, ORDER BY
CREATE UNIQUE INDEX i ON users (email);                   -- unique lookup
CREATE INDEX i ON orders (cust_id, order_date);           -- multi-column,
    prefix rule
CREATE INDEX i ON users (lower(email));                   -- expression
CREATE INDEX i ON events (ts) WHERE event_type = 'purchase';  -- partial
CREATE INDEX i ON orders (cust_id) INCLUDE (amount);      -- covering: Index
    Only Scan
CREATE INDEX i ON events USING brin (ts);                 -- BRIN: huge append
    -ordered column
CREATE INDEX i ON users USING hash (email);               -- hash: equality
    only
```

10 Lab 4: Operators

Technique: Part by part

1. **Read a plan**: the plan of §6. Breakers: Hash build, HashAggregate, Sort.
2. **Three joins**: default Hash Join (≈230 ms, each input once, build 1.3 MB); `enable_hashjoin = off` gives Merge Join (≈430 ms, orders need `external merge Disk: 8184kB`); `enable_mergejoin = off` too gives Nested Loop (≈640 ms, `Index Scan ... loops=20000`). `RESET ALL`.
3. **External sort**: `work_mem = '64kB'`: `external merge Disk: 8192kB`, 390 ms; `'256MB'`: `quicksort Memory: 71680kB`, 190 ms.
4. **Aggregation**: `GROUP BY country` HashAggregate (12 groups); `GROUP BY name` Sort + GroupAggregate (19,000 groups).
5. **Size flips the join**: `c.id < 10` Nested Loop with index scans, 0.26 ms; whole join Hash Join, 230 ms.

Exercises: (1) as part 1; (2) hash < merge < nested loop, because hash reads once, merge sorts 500k orders, nested loop pays 20,000 index descents; (3) the two Sort Method lines, input vs `work_mem`; (4) number of groups decides hash vs sort; (5) $\lceil 200/20 \rceil = 10$ chunks, $200 + 10 \times 2000 = 20{,}200$ (mock 1.5: $300 + 10 \times 3000 = 30{,}300$).

11 Lab 5: Estimates and bad plans

Technique: Part by part

1. **Estimated vs actual**: `amount > 90` estimates 15% of the table; the MA join's size is $|R||S|/\max(d_R, d_S) = 2{,}000 \times 500{,}000/10{,}000 = 100{,}000$. Tabulate `rows=` vs actual per node.
2. **Statistics**: `\x` then the `pg_stats` query; `amount` has ≈10,000 distinct values and a 100-bucket histogram, `order_date` 365.
3. **Selectivity by hand**: `FILTER` fraction ≈ 0.20; `EXPLAIN` shows $0.20 \times N$ because `'MA'` is in the MCV list.
4. **Stale statistics**: insert 300k rows, `EXPLAIN ANALYZE` immediately: estimate stuck at the old size (off by the growth factor); after `ANALYZE orders` it snaps back.
5. **Correlated columns**: sel(country) = sel(city) = 0.2, product 0.04 ⇒ 400 of 10,000 estimated, 2,000 actual; `CREATE STATISTICS s_geo (dependencies) ON country, city FROM customers; ANALYZE customers;` gives ≈ 2,000.

Exercises: (1) accurate means within 2× at every node; (2) they agree to sampling precision; (3) `ANALYZE` rewrote `reltuples` and the histogram, the planner never looks at the table; (4) 400/2,000 before, 2,000/2,000 after; (5) a leaf error multiplies through every join above it and picks the wrong algorithm (a Nested Loop chosen for "40 rows" runs 200,000 times, mock 3.1); an error at the top only mis-prices the last operator.

Part IV

Write-from-memory drill

Mock question	What you must type or read	Practised in
3.1 bad plan	worst estimate, causes, `ANALYZE` + `CREATE STATISTICS`, better join	Lab 5.1, 5.4, 5.5; Lab 4.2
3.2 storage	pages and seconds; Parquet + DuckDB; `pg_relation_size`, `pageinspect`	Ch. 1 demo 4; Ch. 2 §6
3.3 indexes	six `CREATE INDEX`; one that no index helps, with a selectivity	Lab 3.2–3.5
3.4 sort/join cost	runs, passes, I/Os; which join; build side vs `work_mem`	Lab 4.2–4.3, paper exercise
3.5 estimation	independence vs true count; `CREATE STATISTICS`; view vs purchase	Lab 5.3, 5.5
3.6 buffers	hit ratio, ring buffer, `pg_buffercache`	Ch. 1 demo 1 (`BUFFERS`)
3.7 SQL	30-day group-by; skewed `generate_series` insert	Ch. 1 setup; §4

12 Drill

On paper first, then in `adm_lab`. Solutions in §13.

D1. Create `customers` and `orders` with keys; fill 10,000 customers (5 countries) and 50,000 orders (amount 5–105, random day of 2025); refresh statistics.

D2. Insert 1,000,000 `events`, 90% view / 9.5% cart / 0.5% purchase, random timestamp in the last year. (Mock 3.7b)

D3. Purchase events per country in the last 30 days, biggest first. (Mock 3.7a)

D4. Real selectivity of `country = 'MA'` by hand, and the planner's estimate. (Lab 5.3)

D5. The index for each pattern: unique `email`; a `ts` range; `event_type = 'purchase'`; `event_type = 'view'`; `lower(email) = $1`; the join on `user_id`. Which one no index helps, with a number. (Mock 3.3)

D6. Force hash, merge, nested loop on the customers–orders join; then make a sort spill and name the proving line. (Lab 4.2–4.3)

D7. Verify that `events` has about 50 rows per page, two commands. (Mock 3.2c)

D8. Fix the mock 3.1 plan (40 estimated vs 200,000 Casablanca users) and name the join that should appear.

D9. Which tables occupy the buffer pool now. (Mock 3.6c)

D10. Export `events.ts` to Parquet and count per day in DuckDB. (Mock 3.2b)

D11. Find never-used indexes on `events` with sizes ("which indexes to keep", mock 2.3).

D12. Independence estimate for `country = 'MA' AND city = 'Casablanca'` (sel 0.2, 0.1; 2 M users); the true count, and the fix. (Mock 3.5a)

13 Solutions

Solution: D1

The two `CREATE TABLE`s and two `INSERT ... generate_series` of §4, then `ANALYZE;`. Check: `SELECT min(amount), max(amount), min(order_date), max(order_date) FROM orders;`

Solution: D2

The skew insert of §4 (one `random() AS r` in a derived table, CASE cuts at 0.90 and 0.995). Check: `SELECT event_type, count(*) FROM events GROUP BY 1;` expect ≈ 900,000 / 95,000 / 5,000.

Solution: D3

```
SELECT u.country, count(*) AS purchases
FROM   events e JOIN users u ON u.user_id = e.user_id
```

```
WHERE  e.event_type = 'purchase' AND e.ts >= now() - interval '30 days'
GROUP BY u.country ORDER BY purchases DESC;
```

Both predicates are on `events` and get pushed below the join; with the partial index of D5(c) the leaf is an index/bitmap scan.

Solution: D4

```
SELECT count(*) FILTER (WHERE country = 'MA')::float / count(*) FROM
    customers;   -- ~0.20
EXPLAIN SELECT * FROM customers WHERE country = 'MA';     -- rows= ~0.20
    x reltuples (MCV frequency)
```

Solution: D5

```
CREATE UNIQUE INDEX ON users (email);                                -- (a) B
    -tree (hash also works)
CREATE INDEX ON events (ts);                                         -- (b) B
    -tree keeps order; BRIN if append-ordered
CREATE INDEX ON events (ts) WHERE event_type = 'purchase';           -- (c)
    partial: 0.5% = 250,000 rows
CREATE INDEX ON users (lower(email));                                -- (e)
    expression
CREATE INDEX ON events (user_id);                                    -- (f)
    FK side of the join
```

(d) `event_type = 'view'`: selectivity 0.9, 45,000,000 rows on ≈900,000 of 1,000,000 pages. The index threshold is about $1/(4 \cdot 50) = 0.5\%$ of rows; 90% is 180× above it. The planner scans, correctly.

Solution: D6

```
EXPLAIN ANALYZE SELECT * FROM customers c JOIN orders o ON o.cust_id = c.
    id;   -- Hash Join
SET enable_hashjoin = off;   EXPLAIN ANALYZE ...;
          -- Merge Join
SET enable_mergejoin = off;  EXPLAIN ANALYZE ...;
          -- Nested Loop
RESET ALL;
SET work_mem = '64kB';  EXPLAIN ANALYZE SELECT * FROM orders ORDER BY
    amount;  -- Sort Method: external merge Disk
SET work_mem = '256MB'; EXPLAIN ANALYZE SELECT * FROM orders ORDER BY
    amount;  -- Sort Method: quicksort Memory
RESET work_mem;
```

Solution: D7

```
SELECT pg_relation_size('events') / 8192 AS pages,
       (SELECT reltuples::bigint FROM pg_class WHERE relname = 'events')
    AS rows;   -- rows/pages ~ 50
CREATE EXTENSION IF NOT EXISTS pageinspect;
SELECT count(*) FROM heap_page_items(get_raw_page('events', 0));
               -- ~50 on one real page
```

Solution: D8

```
ANALYZE users;  ANALYZE events;                                    --
    stale by 10x
CREATE STATISTICS s_geo (dependencies) ON country, city FROM users;
    ANALYZE users;
```

With 200,000 Casablanca users and 1.2 M purchases known, the planner picks a **Hash Join**: build on the filtered users (≈3,000 pages), one pass over events, instead of 200,000 nested-loop scans of 1,000,000 pages.

Solution: D9, D10

The `pg_buffercache` query and the `COPY ... (FORMAT parquet)` + `read_parquet` pair of Part II. Parquet is cheaper because the file holds one compressed 8-byte column, not 8 GB of full rows with TOAST pointers and jsonb.

Solution: D11

```
SELECT indexrelname, idx_scan, pg_size_pretty(pg_relation_size(indexrelid
    ))
FROM   pg_stat_user_indexes WHERE relname = 'events' AND idx_scan = 0;
```

Keep an index if it is scanned, serves a selective predicate, or enforces uniqueness; each other one costs a leaf write per insert.

Solution: D12

Independence: $0.2 \times 0.1 = 0.02 \Rightarrow 40{,}000$ rows. Truth: Casablanca implies MA, so the conjunction is `city = 'Casablanca'` alone: $0.1 \times 2{,}000{,}000 = 200{,}000$, a $5\times$ under-estimate. Fix: `CREATE STATISTICS s_geo (dependencies) ON country, city FROM users; ANALYZE users;`

Cheat sheet

Need	Command
n rows of anything	`FROM generate_series(1, n) g`
uniform int in $1..k$	`1 + floor(random() * k)::int`
uniform real in $[a, a+w)$, 2 dp	`round((random() * w + a)::numeric, 2)`
random day of a year	`date '2025-01-01' + floor(random() * 365)::int`
random moment, last year	`timestamp '2026-10-12' - random() * interval '365 days'`
categorical uniform	`(ARRAY['a','b','c'])[1 + floor(random() * 3)::int]`
categorical skewed	`CASE WHEN r < .9 THEN ..  WHEN r < .995 THEN .. ELSE ..  END`, `r` from `(SELECT random() AS r FROM generate_series ...)`
last 30 days	`ts >= now() - interval '30 days'`
per-day buckets	`GROUP BY ts::date`
fraction matching p	`count(*) FILTER (WHERE p)::float / count(*)`
plan, estimates only	`EXPLAIN q;`
plan with actuals and pages	`EXPLAIN (ANALYZE, BUFFERS) q;`
force an algorithm away	`SET enable_hashjoin = off;` ... `RESET ALL;`
spill a sort	`SET work_mem = '64kB';`
refresh statistics	`ANALYZE t;`
column statistics	`SELECT * FROM pg_stats WHERE tablename = 't';`
believed row/page counts	`SELECT reltuples, relpages FROM pg_class WHERE relname = 't';`
correlated columns	`CREATE STATISTICS s (dependencies) ON a, b FROM t; ANALYZE t;`
pages of a table	`pg_relation_size('t') / 8192`
rows on a page	`count(*) FROM heap_page_items(get_raw_page('t', 0))`
who is in the pool	`pg_buffercache` joined to `pg_class` on `relfilenode`
unused indexes	`pg_stat_user_indexes WHERE idx_scan = 0`
column to Parquet	`COPY (SELECT c FROM t) TO 'f.parquet' (FORMAT parquet);`
B-tree / unique / multi-col	`CREATE [UNIQUE] INDEX i ON t (a[, b]);`
expression / partial / covering	`ON t (lower(a))` / `ON t (a) WHERE p` / `ON t (a) INCLUDE (b)`
BRIN / hash	`USING brin (ts)` / `USING hash (a)`